HMDA и табличные вычисления

Конспект первой лекции. Все расчёты начинаются с шести записей и затем повторяются на decisions.csv. Поля таблицы объясняются до их использования в командах.

1. Первая рабочая задача data scientist

Сегодня разберём задачу, которую можно решить с помощью таблицы: какую долю ипотечных заявок одобрили? Человек подаёт заявку на покупку жилья, финансовая организация рассматривает её и сообщает решение. Одобрение ещё не означает, что человек воспользовался предложением и получил кредит. Поэтому мы отдельно посчитаем долю одобрений и долю выдач.

Начнём с самих записей: посмотрим, что означает одна строка и как обозначены исходы. Затем загрузим небольшой файл в Python. На шести строках результат можно проверить вручную; когда код станет понятен, применим его к таблице большего размера. В конце сгруппируем записи по организациям и получим сводку для каждой из них.

Для этого используем библиотеку pandas. Предварительных знаний pandas не требуется: будем разбирать команды по мере появления. На практике после перерыва вы самостоятельно загрузите данные, посчитаете обе доли и сохраните расчёт так, чтобы его мог повторить другой человек.

2. Реальные заявки и смысл полей

Перед нами шесть реальных записей об ипотеке. Их источник — HMDA, сокращение от Home Mortgage Disclosure Act, закона США о раскрытии ипотечных данных. В рамках этой отчётности финансовые организации публикуют сведения о заявках и кредитах. Мы используем записи за 2023 год о покупке жилья в округе Колумбия — это Вашингтон.

Прочитаем вторую строку таблицы: в поле action_taken стоит 1, сумма loan_amount — 585 тысяч долларов, доход income — 174 тысячи долларов в год. action_taken означает исход: 1 — кредит выдан; 2 — заявка одобрена, но заявитель не принял предложение; 3 — отказ. Обратите внимание на единицы: сумма хранится в долларах, а доход — в тысячах долларов в год.

source_row — номер строки в сохранённом исходном файле. Он позволяет найти, откуда взят пример. Например, у записи 278 исход 2: предложение одобрено, но выдачи не было.

Для показа мы специально взяли по две записи каждого исхода. Эта маленькая таблица удобна для ручного счёта. Реальную долю в нашем наборе посчитаем позже по всем подходящим записям. В учебном CSV есть ещё несколько полей; весь список увидим при первом чтении файла.

source_row	action_taken	loan_amount, USD	income, тыс. USD/год
2	1	275 000	72
3	1	585 000	174
278	2	475 000	108
304	2	905 000	353
21	3	595 000	184
27	3	1 805 000	667

3. Какие данные будем читать

У нас подготовлены три файла. reading_example.csv содержит шесть строк, которые мы только что разобрали. decisions.csv содержит 6 830 записей с исходами 1, 2 и 3. На этом файле мы будем считать основные показатели.

Третий файл, hmda_2023_dc_raw.csv, — более широкий исходный снимок: 8 575 записей. Кроме трёх знакомых исходов в нём есть код 4 — заявитель отозвал заявку, и код 5 — рассмотрение закрыто из-за неполноты документов. Чтобы не смешивать разные этапы процесса, сегодняшний основной вопрос сформулируем точно: какая доля записей с исходами «выдано», «одобрено, но не принято» и «отказано» относится к одобрениям?

Поэтому общее количество для основного расчёта — 6 830. Пока не считаем процент: сначала научимся читать таблицу и выбирать нужные значения. Все три файла уже лежат в папке data; скачивать их на занятии не нужно. Когда в дальнейшем я скажу «полный набор для расчёта», речь будет именно о decisions.csv. Более широкий исходный файл понадобится для дополнительного задания.

Файл	Строк	Роль
reading_example.csv	6	Разобрать команды вручную
decisions.csv	6 830	Посчитать доли для исходов 1–3
hmda_2023_dc_raw.csv	8 575	Исходный снимок, коды 1–5

4. Четыре типа аналитических вопросов

Теперь понятно, какие записи у нас есть. Различим четыре вопроса, которые можно о них задать.

Описательный вопрос: какую долю заявок одобрили? Ответ получим подсчётом имеющихся исходов. Это основная задача сегодня.

Диагностический вопрос: с чем связано, что у одной организации доля одобрений ниже, чем у другой? Для начала можно сравнить доходы заявителей и запрошенные суммы. Так мы ищем возможные объяснения различий.

Прогнозный вопрос: одобрят ли новую заявку? Здесь уже требуется модель. Проверять её нужно на заявках, которых она не видела при обучении, используя сведения, известные до решения. Причина уже вынесенного отказа подсказала бы ответ, поэтому для такого прогноза она не подходит.

Причинный вопрос: станет ли меньше незавершённых заявок, если упростить анкету? Нужно проверить эффект самого изменения. Например, случайно предложить одним заявителям старую анкету, другим новую и сравнить результаты. В имеющейся таблице такого эксперимента нет.

Сегодня мы научимся получать описательный ответ. Умение посчитать процент будет основой дальнейшей работы, но само по себе не проверяет ни прогноз, ни эффект изменения анкеты.

Тип вопроса	Пример
Описание	Какую долю заявок одобрили?
Диагностика	С чем связано различие организаций?
Прогноз	Одобрят ли новую заявку?
Причинный	Что изменит упрощение анкеты?

5. Зачем нужна pandas

Теперь перейдём к инструменту. pandas — библиотека Python для работы с таблицами. Вместо того чтобы вручную писать цикл по каждой записи, мы сможем обратиться сразу к столбцу, сравнить его значения с условием и посчитать результат.

Основной объект pandas называется DataFrame. Представьте знакомую таблицу: строки содержат записи, столбцы — характеристики. Наши шесть ипотечных записей после чтения CSV будут DataFrame.

Если выбрать один столбец этой таблицы, получится другой объект — Series. Например, столбец action_taken даст последовательность кодов 1, 1, 2, 2, 3, 3. У каждого значения будет метка строки слева. Таблица DataFrame и столбец Series поддерживают разные операции, поэтому мы будем смотреть, какой именно объект возвращает команда.

Сейчас достаточно запомнить эти два названия: DataFrame — таблица, Series — ряд значений, который часто получается при выборе столбца. Сначала импортируем библиотеку и зададим путь к файлам, затем прочитаем наш пример.

6. pd, функция, метод и атрибут

Строка import pandas as pd загружает библиотеку и даёт ей короткое имя pd. Поэтому функцию чтения мы будем вызывать как pd.read_csv. Функция выполняет действие; круглые скобки означают вызов и содержат параметры.

Когда действие принадлежит конкретной таблице, его называют методом. Например, table.head() просит таблицу показать первые строки. Атрибут хранит характеристику объекта: table.shape сообщает размер таблицы. Здесь круглые скобки не нужны. Эти команды подробно разберём после чтения файла.

В начале ноутбука есть и подготовка пути. Path из библиотеки pathlib помогает работать с именами папок и файлов. В переменной DATA сохраним путь ../data: две точки означают подняться на уровень выше, затем перейти в data. Ноутбук лежит в teacher, а data находится рядом с этой папкой.

assert DATA.is_dir() проверяет, существует ли папка. assert — команда Python: если условие истинно, выполнение продолжается; если ложно, появляется ошибка. Если здесь остановились, сначала проверяем расположение папок. Данные ещё не

прочитаны. Импорт numpy под именем np понадобится во второй части лекции; сейчас перейдём к pandas.

7. CSV и первое чтение read_csv

CSV — текстовый файл, в котором поля разделены запятыми. Первая строка содержит имена столбцов, следующие строки — значения записей. На экране перечислены все семь полей нашего учебного файла.

Четыре поля уже знакомы: source_row — номер исходной строки, action_taken — исход, loan_amount — сумма в долларах, income — доход в тысячах долларов в год. Ещё три поля: activity_year — отчётный год, lei — код финансовой организации, county_code — код территории. В наших записях год равен 2023, а код округа — 11001. Код организации может повторяться: одна организация рассматривает много заявок.

В выражении DATA / "reading_example.csv" косая черта соединяет путь к папке с именем файла. Этот путь передаётся функции read_csv. Она читает файл, создаёт DataFrame, и знак равенства сохраняет таблицу в переменную example.

Теперь выполним строку и выведем example. Слева Jupyter покажет метки строк от 0 до 5. Это отдельная служебная часть отображения таблицы; она не добавляет восьмой столбец в CSV. В самой таблице видны семь полей из нашего списка.

Поле	Что хранится
source_row	Номер строки исходного CSV
activity_year	Отчётный год: 2023
lei	Код финансовой организации
county_code	Код территории: 11001
action_taken	Исход: 1, 2 или 3
loan_amount	Сумма, USD
income	Доход, тысячи USD в год

8. Параметры read_csv

Первое чтение сработало с настройками по умолчанию: pandas нашла запятые и использовала первую строку как имена столбцов. Поведение функции можно менять параметрами.

sep задаёт разделитель полей. dtype позволяет указать тип хранения конкретных столбцов. Например, код территории можно читать как строку, даже если он состоит из цифр. nrows ограничивает количество читаемых записей. usecols выбирает, какие столбцы загрузить.

Это разные настройки одного действия — чтения файла. Они не меняют сам CSV на диске, а определяют, какую таблицу мы получим в памяти Python. Не нужно запоминать всю сигнатуру read_csv. Разберём по одному параметру и каждый раз посмотрим, что изменилось в результате.

Начнём с разделителя. Мы знаем, что в нашем файле запятые. Специально укажем вертикальную черту и посмотрим, заметит ли Python ошибку автоматически.

9. Неверный sep и диагностика

Читаем тот же файл, но передаём sep="|". Результат сохраняем в отдельную переменную wrong_sep, чтобы правильная таблица example осталась доступной.

Посмотрим на вывод. Код отработал без исключения, но вместо семи столбцов получился один длинный текстовый столбец. pandas искала вертикальные черты и не нашла их, поэтому не разделила строку на отдельные поля. Даже имена всех семи полей оказались одним именем столбца.

Отсюда полезная привычка: после чтения файла посмотреть, что получилось. Отсутствие ошибки Python означает только, что функция смогла выполнить запрос. Совпадает ли результат с устройством исходного файла, проверяем мы.

Чтобы прочитать этот CSV правильно, нужно оставить sep по умолчанию или явно поставить запятую. Исходный файл при обоих вызовах остался прежним. Позже сравним размеры example и wrong_sep численно.

10. dtype и сохранение идентификаторов

Посмотрим на два поля, которые уже видели в таблице: lei и county_code. Оба обозначают объекты: организацию и территорию. Считать среднее арифметическое кодов территорий нам не нужно. Поэтому удобно хранить их как строки.

Параметр dtype принимает словарь. Ключ — имя столбца, значение — желаемый тип. В нашем вызове для lei и county_code указан string. Остальные поля pandas распознает сама. Заново прочитаем файл с этим параметром и сохраним в example.

Код территории 11001 внешне похож на число, но строковый тип помогает одинаково обрабатывать коды, в том числе содержащие начальные нули в других территориях. Содержимое наших шести записей от этого не меняется.

Столбец action_taken оставляем числовым: позже будем сравнивать его значения с числами 1, 2 и 3. Здесь важно различать смысл поля и тип хранения. action_taken хранится как целое число, но эти числа обозначают категории исходов. Складывать сами коды ради банковского показателя мы не будем: сначала построим условие нужного исхода.

11. nrows и usecols

Теперь ограничим чтение по двум направлениям. В первом вызове nrows=3 просит загрузить только первые три записи. Заголовок при этом остаётся заголовком и не считается записью. Результат preview содержит три строки и все семь столбцов.

Во втором вызове usecols получает список двух имён: action_taken и income. Здесь ограничиваем столбцы, поэтому получаем шесть строк, но только два поля. Сохраняем результат в only_fields.

Посмотрим обе таблицы и сравним их с example. example по-прежнему содержит шесть строк и семь столбцов: мы не присваивали ей результат этих двух вызовов. Каждый вызов создаёт свой объект.

Такое частичное чтение полезно, когда файл большой: можно сначала посмотреть несколько записей или загрузить только необходимые поля. Но показатель, посчитанный по preview, описывал бы лишь три загруженные записи. Для основного расчёта позднее прочитаем decisions.csv целиком.

12. Размер, имена и типы столбцов

Мы сравнили таблицы глазами. Теперь получим их характеристики командами. example.shape возвращает пару чисел: сначала количество строк, затем количество столбцов. Ожидаем (6, 7). У wrong_sep будет (6, 1), а у preview — (3, 7).

columns возвращает имена столбцов. Это удобно, когда нужно проверить написание поля перед выбором: Python различает точные имена. dtypes показывает тип хранения каждого столбца. Посмотрим, что у lei и county_code строковый тип, а action_taken — целочисленный. У суммы и дохода в этом файле может быть числовой тип с дробной частью: это видно по обозначению float.

Все три обращения записываются без круглых скобок. Мы читаем атрибут — характеристику уже существующего объекта. Когда дальше будем вызывать head() или info(), скобки понадобятся, потому что там выполняется действие.

После этих проверок у нас есть ответ на три базовых вопроса о загруженной таблице: сколько в ней записей, какие есть поля и как Python хранит их значения. Следующий шаг — выбрать столбец, по которому будем считать исходы.

13. Выбор одного столбца и Series

Нас интересует исход каждой записи. В таблице он хранится в action_taken. Запишем имя столбца в квадратных скобках после example и сохраним результат в outcomes.

Выведем тип объекта: получился pandas Series. Ниже видны шесть значений — 1, 1, 2, 2, 3, 3 — и метки строк слева. Это те же исходы, которые были в нашей маленькой таблице. Мы просто выбрали один столбец для дальнейшей работы.

Имя outcomes выбрали мы сами; оно означает «исходы». pandas ничего не узнаёт из названия переменной: нужные данные выбраны именно выражением example["action_taken"]. Переменная example всё ещё ссылается на полную таблицу из семи столбцов.

В ближайших шагах будем применять методы к outcomes. Поэтому полезно удерживать, с чем работает каждая строка: example — DataFrame, outcomes — Series. Одни квадратные скобки и строка с именем поля здесь вернули столбец.

14. Список столбцов и двойные скобки

Если нужно несколько столбцов, передадим список имён. Сначала создаём обычный список Python names, в нём action_taken, income и loan_amount. Затем пишем example[names]. Результат selected — таблица DataFrame из шести строк и трёх столбцов.

Две пары квадратных скобок появляются, когда список записывают прямо внутри выбора: example[["action_taken", "income", "loan_amount"]]. Внутренние скобки создают список, внешние выбирают по этому списку столбцы таблицы. Это та же операция, что с отдельной переменной names.

Разница заметна и при одном поле. example["action_taken"] возвращает Series. example[["action_taken"]] получает список из одного имени и возвращает DataFrame с одним столбцом. Значения исходов одинаковые, но тип результата разный.

Сейчас покажем selected и его тип. Для дальнейших условий сохраним outcomes как Series, а для просмотра связанных полей будем выбирать таблицу. Исходная example от этих выборов не уменьшилась.

15. head и цепочка выбора

Когда таблица большая, выводить её целиком неудобно. Метод head возвращает первые строки: без параметра — до пяти, с числом 3 — до трёх. Выполним example.head(3). Получим первые три строки, но в самой example останутся все шесть.

Сравним с nrows=3. Там функция читала из файла только три записи. Здесь файл уже прочитан полностью, а head создаёт небольшой результат для просмотра. Поэтому после example.head(3) размер example всё ещё (6, 7).

Теперь соединим знакомые операции: example[["action_taken", "income"]].head(6). Сначала выбор столбцов возвращает таблицу с исходом и доходом. Затем head(6) берёт из неё первые шесть строк. Цепочку читаем слева направо.

Посмотрим на вывод и сопоставим доход с исходом в каждой записи. Если позже увидите длинную цепочку команд, её можно временно разложить на переменные и посмотреть каждый промежуточный объект. Это помогает понять, какая операция изменила столбцы, а какая выбрала строки.

16. info, isna и неизвестные значения

Метод info() печатает краткий отчёт о таблице. В нём видны количество строк, имена полей, число непустых значений и типы. Выполним example.info() и сравним отчёт с тем, что уже узнали через shape и dtypes.

Обратите внимание: info печатает отчёт, а не возвращает новую таблицу. Поэтому запись example = example.info() была бы ошибкой: вместо DataFrame в переменную попал бы результат None.

Теперь выберем income и вызовем isna(). Для каждого значения дохода получим True, если оно отсутствует, и False, если есть. Такой ряд ответов называют логическим Series. В маленьком примере все шесть доходов известны, поэтому увидим только False. В большом наборе дальше встретим и пропуски.

Неизвестный доход и нулевой доход — разные ситуации. Когда будем считать долю одобрений, нам понадобится поле исхода, а доход в саму формулу не входит. Поэтому позже сначала посмотрим, какие поля известны, и только затем решим, каких данных достаточно для нашего расчёта.

17. value_counts и частоты категорий

В outcomes лежат коды исходов. Метод value_counts() подсчитывает, сколько раз встретился каждый код. Выполним outcomes.value_counts(dropna=False). В результате слева будут сами коды, справа — их количества. У каждого из кодов 1, 2 и 3 в нашем примере по две записи.

Параметр dropna=False просит включить в подсчёт пропуски, если они есть. По умолчанию pandas их не показывает. Поэтому при первичном знакомстве с данными удобно явно включать их отображение.

Ещё один параметр, normalize=True, вместо количества вернул бы долю каждого кода. Две записи из шести — это примерно 0,3333. Чтобы получить процент, умножаем на сто: получается 33,33 процента.

Теперь можем проверить ручное наблюдение: всего шесть исходов, каждого вида по два. Но для доли одобрений одного кода недостаточно. В неё входят и выданные кредиты с кодом 1, и одобрения без принятия предложения с кодом 2. На следующем слайде соберём эти два исхода вместе.

18. Метрики и ручной расчёт

Пока отложим Python и посчитаем по таблице. Для каждой записи справа отмечено, входит ли она в одобрения и входит ли в выдачи. True означает «да», False — «нет».

Одобрение включает коды 1 и 2. В наших шести строках это четыре записи. Выдача включает только код 1, таких записей две. Общее количество в обоих расчётах — шесть.

Запишем дроби: доля одобрений — четыре разделить на шесть; доля выдач — два разделить на шесть. Число сверху называют числителем: сколько записей подходит под условие. Число снизу — знаменателем: среди какого общего количества записей считаем долю. Доли равны примерно 0,6667 и 0,3333, или 66,67 и 33,33 процента.

Эти числа относятся к шести показанным строкам. Мы специально подобрали по две записи каждого исхода, поэтому сейчас проверяем способ расчёта. В дальнейшем сделаем те же действия на 6 830 записях. Сначала научим Python ставить для каждой строки те же отметки, которые видны в таблице.

Код	Одобрение?	Выдача?
1	True	True
1	True	True
2	True	False
2	True	False
3	False	False
3	False	False

19. eq и isin условия для каждой строки

Метод eq означает проверку равенства. outcomes.eq(1) сравнивает каждый код исхода с числом 1. Первые две записи имеют код 1, поэтому получат True; остальные четыре — False. Сохраним этот результат в issued_flag, то есть признак выдачи.

Для одобрений нужно проверить сразу два возможных значения. Метод isin получает список допустимых кодов. outcomes.isin([1, 2]) спрашивает для каждого исхода: входит ли он в список из единицы и двойки? Результат approved_flag содержит четыре True и два False.

Запустим обе команды и сравним полученные ряды с таблицей на слайде. Исходных записей было шесть, и ответов в каждом логическом Series тоже шесть. Пока мы не удалили ни одной строки и не получили количество одобрений: для каждой строки только записали ответ на условие.

Имена issued_flag и approved_flag — обычные переменные Python. В них теперь лежат готовые логические результаты. Следующим действием посчитаем, сколько раз встретилось True, и получим числители наших долей.

Код	eq(1)	isin([1, 2])
1	True	True
2	False	True
3	False	False

20. sum, len и проверяемая доля

В логическом Series метод sum() считает True: при таком суммировании True даёт единицу, False — ноль. Поэтому approved_flag.sum() возвращает 4, а issued_flag.sum() — 2. Это наши числители.

Функция len(example) возвращает количество строк таблицы — 6. Сохраняем его в n_decisions. Теперь делим n_approved на n_decisions, а n_issued — на тот же n_decisions. Результаты сначала являются долями от нуля до единицы. В строке вывода умножаем их на сто. Запись :.2f внутри форматированной строки показывает два знака после десятичной точки; сам расчёт от этого не округляется заранее.

В коде есть assert n_decisions > 0. Знакомая команда assert проверяет условие перед делением: общее количество должно быть положительным. В конце проверяем, что доля выдач не больше доли одобрений и обе находятся между нулём и единицей. Если условие ложно, выполнение остановится с AssertionError.

Эти проверки помогают заметить сбой, а правильность выбранных кодов мы уже проверили вручную по шести строкам. Теперь можно повторить понятный расчёт на большем файле.

21. Полный срез 6 830 решений

Переходим от подобранного примера к decisions.csv — тому самому набору из 6 830 записей, который назвали в начале. Читаем его в новую переменную decisions. Маленькая таблица example остаётся доступной отдельно.

Сначала посмотрим info и частоты исходов. В поле action_taken нет пропусков. Выдач с кодом 1 — 5 956, одобрений без принятия предложения с кодом 2 — 183, отказов с кодом 3 — 691. Их сумма равна 6 830. В income есть 342 пропуска; для сегодняшней доли достаточно известного исхода каждой записи.

Повторяем знакомые условия. Для одобрений берём коды 1 и 2: получаем 6 139. Делим на 6 830 и переводим в проценты — 89,88 процента. Для выдач берём только код 1: 5 956 из 6 830, или 87,20 процента.

Итак, 89,88 процента — это доля одобрений среди выбранных записей о покупке жилья в Вашингтоне за 2023 год с исходами 1–3. Разница с долей выдач составляет 2,68 процентного пункта. Она соответствует 183 одобрениям, после которых предложение не было принято. Процентный пункт здесь означает обычное вычитание двух процентов.

22. Ошибка определения и ошибка типа

Разберём две ошибки, которые могут дать внешне правдоподобный результат. Первая: написать outcomes.eq(1), а результат назвать долей одобрений. Команда выполнится, но посчитает только выдачи. Исправление зависит от вопроса: для одобрений используем isin([1, 2]), а для выдач оставляем eq(1) и правильное название.

Вторая ошибка связана с типом. Создадим отдельную таблицу text_copy и при чтении попросим хранить action_taken как string. Теперь единицы в этом столбце — текстовые значения "1". Метод eq(1) сравнивает их с числом 1 и вернёт False. Сумма True окажется нулевой, хотя записи о выдачах никуда не исчезли.

Сравним с eq("1"): теперь тип значения совпадает, и увидим две выдачи маленького примера. Изменения затронули только отдельную text_copy. В example и decisions исходы по-прежнему числовые.

Если получили неожиданное число, полезно проверить по порядку: какую метрику хотели, какое условие написали, какие типы сравниваем и сколько True получилось до деления. Название переменной само по себе ничего не проверяет.

23. Индекс метка и позиция

До сих пор выбирали столбцы. Для выбора конкретной строки нужно различать её метку и позицию. В исходной example слева стоят метки 0, 1, 2, 3, 4, 5. Они совпадают с позициями строк, поэтому разница пока незаметна.

Вспомним source_row: это номер строки исходного CSV, который видели в первой таблице. Сделаем его индексом с помощью example.set_index("source_row") и сохраним результат в indexed. Теперь метками строк станут 2, 3, 278, 304, 21 и 27. Значения source_row переместились из обычного столбца в индекс.

Рассмотрим запись с меткой 278. Она третья сверху, поэтому её позиция — 2: позиции в Python считаются с нуля. Метка этой строки — 278, позиция — 2. Это два разных способа указать одну запись.

example при этом осталась прежней: новую таблицу сохранили в indexed. В следующем шаге выберем исход записи 278 сначала по метке, затем по позиции и сравним результаты.

24. loc и iloc

У таблицы pandas есть два способа адресовать строки и столбцы. loc выбирает по меткам: indexed.loc[278, "action_taken"] означает строку с меткой 278 и столбец с именем action_taken. Мы видели эту запись раньше: у неё исход 2 — одобрено, но не принято.

iloc выбирает по позициям. В indexed нужная строка третья сверху, её позиция 2. После переноса source_row в индекс обычные столбцы идут так: activity_year, lei, county_code, action_taken, loan_amount, income. Значит, action_taken — четвёртый столбец, позиция 3. Поэтому indexed.iloc[2, 3] вернёт то же число 2.

Перед выполнением посмотрим indexed.columns и убедимся в порядке столбцов. Затем вызовем оба выражения и проверим равенство через assert.

Если переставить строки или столбцы, позиционные номера могут измениться. Поэтому для выбора по известному имени поля удобно использовать loc. iloc полезен, когда задача сформулирована именно через место в таблице: например, первые строки или конкретная позиция. loc и iloc записываются с квадратными скобками.

25. Фильтрация строк

Теперь выберем не одну строку, а все отказы. Работаем с большой таблицей decisions. Условие decisions["action_taken"].eq(3) возвращает по одному True или False для каждой из 6 830 записей. True стоит там, где исход равен 3.

Сохраним этот логический Series в denied_mask. Такую последовательность ответов называют маской: она указывает, какие строки оставить. В loc можно передать маску вместо отдельной метки строки.

Запись decisions.loc[denied_mask, ["source_row", "lei", "loan_amount", "income"]] выбирает строки с True и четыре перечисленных поля. Все эти поля мы уже видели в словаре: исходный номер, организация, сумма и доход. Результат сохраним в denied.

После выполнения получим 691 строку и четыре столбца. Это согласуется с количеством отказов, которое уже посчитали на слайде 21. Посмотрим первые пять строк. Исходная decisions всё ещё содержит 6 830 записей: фильтрация вернула отдельную таблицу. Теперь можем изучать суммы именно отказанных заявок.

26. Несколько условий

У нас уже есть denied_mask: True для отказов. Добавим второе условие — сумма больше миллиона долларов. Метод gt означает «больше» и даёт тот же результат, что знак >. Выражение decisions["loan_amount"].gt(1_000_000) вернёт логический Series, который сохраним в large_mask.

Подчёркивания в 1_000_000 нужны для чтения. Это возможность самого Python: число равно 1000000. pandas получает обычное целое число.

Нужно оставить заявки, которые одновременно крупные и отказанные. Запишем denied_mask & large_mask. Оператор & берёт два ответа для одной строки и возвращает True, только если оба True. Например, отказ есть, но сумма меньше миллиона: True & False даёт False. Отказ есть и сумма больше миллиона: True & True даёт True. Оператор | дал бы True, если выполнено хотя бы одно условие, а ~ меняет True на False и наоборот.

Обычный and здесь не подходит. Он требует один ответ для каждого операнда, а в наших Series тысячи ответов. Как их свести к одному — достаточно ли хотя бы одного True или нужны все — ему не задано. Для построчного объединения используем &.

Полученную маску передадим в loc, затем посмотрим выбранные суммы и исходы. Если писать сравнения прямо в одной строке через &, каждое сравнение нужно заключить в скобки.

27. Сортировка и пять крупнейших сумм

В denied находятся все 691 отказанная заявка. Хотим увидеть пять самых крупных запрошенных сумм. Для этого сначала отсортируем таблицу по loan_amount, а затем возьмём первые пять строк.

Метод sort_values получает список столбцов, по которым упорядочивает записи. Первое поле — loan_amount, направление ascending=False означает убывание: большие суммы окажутся сверху. Второе поле — source_row, для него ascending=True: если суммы совпадут, записи будут упорядочены по исходному номеру по возрастанию.

Результат сохраняем в denied_sorted и вызываем head(5). Теперь первые пять строк действительно относятся к самым крупным суммам среди отказов нашего набора. Если бы вызвали head(5) до сортировки, получили бы просто пять первых строк в текущем порядке.

Сортировка не удаляет записи. Проверим, что количество строк осталось тем же. Атрибут is_monotonic_decreasing сообщает, идёт ли столбец по невозрастанию; assert проверяет это для loan_amount. Так мы контролируем не конкретную первую сумму, а требуемое свойство результата.

28. Вычисляемый столбец и единицы

На первом примере доход 108 означал 108 тысяч долларов в год. Для дальнейшей работы переведём доходы в доллары и дадим новому полю явное имя income_usd.

Сначала prepared = decisions.copy() создаёт отдельную копию таблицы. Затем в prepared["income_usd"] записываем prepared["income"] * 1000. pandas умножает каждое значение столбца на тысячу и сохраняет результат в новом столбце. Отдельный цикл по строкам писать не нужно.

Выведем рядом income и income_usd для первых пяти записей. Например, значение 72 должно превратиться в 72 000. Старый income остаётся в исходных единицах, а имя нового столбца подсказывает единицу результата.

В большой таблице мы уже нашли 342 отсутствующих дохода. Умножение не создаёт для них известного значения: в новом поле они тоже останутся пропусками. Проверим их количество через isna().sum(): isna ставит True на пропусках, а sum считает эти True. Также проверим, что в исходной decisions столбца income_usd нет: добавляли его именно в копию prepared.

29. NumPy и векторные вычисления

Похожую операцию можно выполнить с помощью NumPy — библиотеки для вычислений с массивами. Массив здесь представляет последовательность значений. Термин «векторная операция» означает, что мы записываем умножение один раз, а оно применяется ко всем элементам.

Возьмём знакомый столбец income и вызовем to_numpy(). Результат сохраним в income_array. Это объект ndarray: значения доходов остались, но отдельные метки строк и имя столбца из Series в массив не перешли.

Посмотрим type, shape и dtype. shape сообщает размер массива, dtype — тип его элементов. Срез [:6] показывает первые шесть значений. Теперь income_array * 1000 умножит каждое число на тысячу, как только что делала pandas.

В массиве отсутствующие числовые значения обозначены NaN. Функция np.isnan создаёт признаки пропусков, и sum снова считает их. В конце np.allclose сравнивает два вычисленных результата с небольшой допустимой погрешностью; equal_nan=True считает совпавшие пропуски равными для этой проверки.

Сейчас достаточно увидеть связь: pandas удобна для именованных таблиц, NumPy позволяет работать с их числовыми значениями как с массивами.

30. groupby и новая единица наблюдения

До этого одна строка decisions означала одну опубликованную ипотечную запись. Теперь хотим получить сведения по каждой организации. Для этого используем lei — код организации, который рассмотрели при первом чтении файла.

Команда decisions.groupby("lei", dropna=False) собирает записи с одинаковым кодом в группы. Пока это объект группировки, а не готовая итоговая таблица. Параметр dropna=False означает, что при наличии пропущенного кода такая группа тоже учитывалась бы.

Вызов groups.size() считает число записей внутри каждой группы. В результате слева будут коды организаций, справа — количества. Теперь одна строка результата относится к организации. Проверим, что сумма всех этих количеств снова равна 6 830: при группировке учли все записи.

Можно выбрать числовой столбец loan_amount и вызвать mean(). Тогда для каждой организации получим среднюю сумму её записей. Среднее получается как сумма известных значений, делённая на их количество. При наличии пропусков mean обычно пропускает их.

Посмотрим первые строки двух результатов: количество записей и среднюю сумму. В следующем шаге соберём несколько показателей в одну таблицу, чтобы не хранить каждую сводку отдельно.

31. agg и профиль организаций

Работаем с prepared — копией большой таблицы. Добавим is_approved: для каждой записи это знакомая проверка action_taken.isin([1, 2]). Такой новый столбец содержит True для одобрений и False для отказов.

Теперь группируем по lei и вызываем agg. Внутри перечислены имена будущих столбцов и пары «исходное поле, операция». n_decisions=("action_taken", "size") означает посчитать записи группы. n_approved=("is_approved", "sum") — сложить логические признаки, то есть получить число одобрений. mean_loan_usd=("loan_amount", "mean") — вычислить среднюю сумму.

reset_index() возвращает код организации из индекса в обычный столбец итоговой таблицы. Результат назовём profile и покажем первые пять строк. У каждой организации теперь рядом стоят её количество записей, число одобрений и средняя сумма.

Добавим approval_rate: n_approved разделить на n_decisions. Это доля внутри конкретной организации. Если хотим восстановить долю для всех записей, сначала складываем количества одобрений всех организаций, затем делим на сумму их количеств записей. Простое среднее долей дало бы маленькой организации такой же вес, как большой. В проверках увидим знакомые общие суммы — 6 139 и 6 830.

32. Интерпретация и передача результата

Сформулируем результат так, чтобы другой человек понял, что именно мы посчитали: среди 6 830 опубликованных записей о покупке жилья в округе Колумбия за 2023 год с исходами 1–3 одобрены 89,88 процента; выдачей завершились 87,20 процента. Одобрениями считали коды 1 и 2, выдачами — код 1.

В этой фразе есть и число, и группа записей, к которой оно относится. Количество 6 830 важно сохранить рядом с процентом. При сравнении организаций также сохраняем их количества записей: одинаковые проценты могут быть посчитаны по очень разным объёмам данных.

Для передачи работы сохраним ноутбук с кодом и README — небольшой текстовый файл с вопросом, источником, определениями и инструкцией запуска. Карточка данных уже лежит рядом с CSV: в ней можно проверить значения полей и происхождение снимка.

В Jupyter команда Restart Kernel очищает переменные из памяти, а Run All выполняет ячейки сверху вниз. Такой запуск проверяет, что результат можно получить из сохранённого кода, без случайных действий, которые остались только в памяти. На практике это будет последним шагом перед сохранением работы в Git.

33. Закрепление и переход к практике

Мы прошли весь расчёт: увидели записи, разобрали их поля, прочитали CSV и проверили структуру. Затем выбрали исходы, построили условия, посчитали подходящие записи и разделили на общее количество. На маленьком примере проверили результат вручную, после этого повторили действия на большом наборе.

Во второй части лекции научились выбирать строки, объединять условия, сортировать суммы, создавать числовой столбец и собирать записи по организациям. На первой практике главным будет уверенно выполнить основной расчёт. Фильтры и группировку дополнительно отработаем на отдельной практике P02.

После перерыва откройте рабочий ноутбук первой практики. Формулировки заданий находятся в методичке, а в ноутбуке оставлены заголовки и код. Вам предстоит проверить чтение файла, дописать расчёты одобрений и выдач и выполнить проверки. Определения показателей и короткий вывод запишите в README.

В конце перезапустим ядро, выполним все ячейки по порядку и сохраним работу локальным коммитом Git. Для этого занятия не требуется публиковать решение в интернете. Сейчас перерыв, затем начнём практику с проверки окружения.

Повторение

Изученные методы (references.md) · Словарь и происхождение данных (../data/card.md).